Devoid Viewer

Codebase & Installer Documentation

This document explains every file, struct, method, and design decision in the Devoid Viewer codebase, and documents the WiX installer. It is written for someone new to Rust who wants to understand not only *what* the code does but *why* it was written that way.

Table of Contents

Section	Topic	Page
1	Project structure overview	2
2	Cargo.toml — dependencies explained	2
3	nav.rs — directory navigation	5
4	loader.rs — image decoding	7
5	main.rs — application core	9
6	Threading & asynchronous design	19
7	Key design decisions & trade-offs	21
8	Caveats & known limitations	23
9	Installer — main.wxs	24
10	Glossary of Rust concepts used	27

1. Project Structure Overview

The project follows the standard Cargo layout:

File	Responsibility
`Cargo.toml`	Project name, version, and all external dependencies.
`build.rs`	Build script — embeds devoid.ico and version metadata into the exe via winres.
`src/main.rs`	Event loop, app state, input handling, pixel renderer, window management.
`src/loader.rs`	Image decoding for all supported formats. Owns Frame and DecodedImage.
`src/nav.rs`	Directory scanning, human-sorted sibling list, next/prev navigation.
`wix/main.wxs`	WiX installer definition — features, file associations, shortcuts.
`devoid.ico`	Application icon. Multi-size (16, 32, 48, 256px). Used by the exe, window title bar, taskbar, and installe

2. Cargo.toml — Dependencies Explained

Each crate was chosen deliberately for leanness. The dependency graph is roughly 50–70 crates and the release binary around 2–3 MB, compared to 200+ crates and ~9 MB with a full GUI toolkit.

Windowing: winit

winit is the de-facto standard Rust crate for OS windows and input events. It handles the event loop, keyboard, mouse, window resize, drag-and-drop, and monitor queries. The rwh_05 feature enables the raw-window-handle v0.5 interface that softbuffer requires. On Windows we also use the platform extension EventLoopBuilderExtWindows to install a Win32 message hook.

Pixel display: softbuffer

softbuffer provides a Vec<u32> mapping directly to the window's pixels. No GPU pipeline, no shader compilation. On Windows it uses a GDI DIB section, compositor-accelerated without a GPU context.

> ■ *Known limitation: DWM fills every new window with white before the application paints. Mitigated by: (1) with_visible(false) when an image argument is provided, and (2) painting gray synchronously before the event loop starts. The window the user sees is always gray or the image.*

Native dialogs: windows-sys

windows-sys is a zero-overhead Windows API binding. Used for MessageBoxW (delete/error dialogs), SystemParametersInfoW(SPI_GETWORKAREA) (exact taskbar exclusion), and PostMessageW (taskbar thumbnail close fix). Target-specific — does not affect non-Windows builds.

Image decoding: image

The **image** crate supports BMP, PNG, JPEG, GIF, WebP, TIFF, ICO. Features enabled selectively. GIF animation uses image::codecs::gif::GifDecoder and IntoFrames. The ICO format is also used to decode devoid.ico for the window icon.

JPEG XL: jxl-oxide

JPEG XL is not yet in the image crate. Pure-Rust decoder using the low-level streaming API.

SVG: resvg + usvg + tiny-skia

- **usvg** parses and normalises the SVG scene tree.
- **resvg** renders into a tiny-skia Pixmap.
- **tiny-skia** rasterises to premultiplied RGBA, which we unpremultiply.

Channel communication: crossbeam-channel

Faster and more ergonomic than std::sync::mpsc. Bounded at capacity 32 to throttle background threads if the UI falls behind.

Windows resource embedding: winres (build dependency)

Runs at build time only. Embeds devoid.ico and version metadata into the .exe. Zero runtime cost.

3. nav.rs — Directory Navigation

Maintains a human-sorted list of sibling image files in the same directory as the currently open file. Navigation wraps at both ends.

struct DirNavigator

- **files: Vec<PathBuf>** — sorted sibling list.
- **index: usize** — position of the current file.

Derives **Default** — auto-generated empty initialiser.

fn set_current(path)

Called on every image open. Rebuilds the sibling list only when the directory changes. Falls back to a single-entry list if the path is not found in the scan.

fn rebuild(dir)

Reads the directory, filters for supported files, sorts using **human_sort** so img2 < img10 rather than img10 < img2.

fn next() / fn prev()

Return None if fewer than 2 images exist — navigation on a single-image directory is a no-op, preventing a redundant reload. Both wrap around and use checked_sub(1) for safe unsigned arithmetic.

fn peek_next() / fn peek_prev()

Read-only lookahead used by preload_adjacent(). Also returns None for single-image directories.

fn remove(path)

Removes a path after deletion. Uses saturating_sub to keep the index in bounds.

fn human_sort / fn consume_number

Digit runs compared numerically: consume_number() reads the full run as u64, ensuring img2 < img10.

4. loader.rs — Image Decoding

Owns the image type definitions and all decoding logic. Returns **DecodedImage** directly — no conversion needed in main.rs.

struct Frame

pixels: Vec<u8> — straight RGBA8, row-major. **delay_ms: u32** — frame duration (0 for non-animated formats).

struct DecodedImage

One or more frames sharing the same dimensions. **is_animated()** — true when frames.len() > 1. **frame(idx)** — reference to a specific frame. **single()** — private convenience constructor.

fn load_image(path) → LoadResult

Dispatches by lowercased extension: svg → load_svg, jxl → load_jxl, gif → load_gif, everything else → load_standard.

fn load_standard — BMP, PNG, JPEG, WebP, TIFF, ICO

image::open() decodes and normalises to RGBA8.

fn load_gif — Animated GIF

Uses GifDecoder with AnimationDecoder. Dimensions read before consuming into frames — single file open. Per-frame delay from frame.delay().numer_denom_ms() with 20ms minimum and 100ms fallback.

fn load_jxl — JPEG XL

- BufReader for efficient I/O.
- request_color_encoding(srgb(...)) forces sRGB output.
- render_frame(0) decodes frame 0.
- Stream yields interleaved f32. Channels 1/2/3/4 expanded to RGBA u8 via f32_to_u8 (clamps then scales).

fn load_svg — SVG

- usvg parses and normalises the scene tree.
- resvg renders to a tiny-skia Pixmap (premultiplied RGBA).
- Unpremultiply: divide each channel by alpha. Zero-alpha pixels zeroed.

5. main.rs — Application Core

Contains the winit event loop, all application state, and the pixel renderer. No widget toolkit, no font renderer, no GPU pipeline.

5.1 const BG: u8 = 42

Background grey for the blank state, alpha compositing, and the pre-loop synchronous gray paint. Used in render(), composite(), and the startup surface fill via pack_rgb(BG, BG, BG).

5.2 enum Msg

Four variants sent from background threads to the UI thread:

- **Loaded** — primary decode complete: path, image, resize flag.
- **Preloaded** — speculative decode for the cache.
- **Error** — decode failed: path and error string.
- **Deleted** — async deletion result: Ok(()) or Err(String).

All contained types are Send automatically.

5.3 struct WakeUp / type Proxy

WakeUp is a zero-size marker injected by background threads via proxy.send_event(WakeUp) to wake the sleeping event loop. **Proxy** is a type alias for the verbose EventLoopProxy generic.

5.4 struct View<'a>

A snapshot of what the renderer needs: image reference, current frame index, and pan offset. App::view() produces one each frame. The 'a lifetime ensures View cannot outlive App. No zoom field — the viewer always uses fit-to-window scale.

5.5 enum OpenMode

FirstImage — resize the window to the image. **Navigate** — keep the existing window size.

5.6 struct App — fields

- **current_image** — not cleared when navigation starts. The previous image stays visible until the new one is decoded.
- **frame_index / next_frame_at** — GIF animation state.
- **pan / drag_start / pan_origin** — pan state. Reset on each new image.
- **loading_path** — path being decoded. Used to discard stale channel messages if the user navigated away.
- **preload_cache** — decoded neighbours in CPU memory.
- **deleting** — prevents double-deletion in flight.

- **pending_center** — triggers centering on the event after a resize has been applied (two-event sequence).

5.7 fn open_image()

Cache hit → apply_image directly. Otherwise sets loading_path, updates the navigator, spawns a background thread that calls load_image(), posts a Msg, and fires WakeUp.

5.8 fn apply_image() → Option<(u32, u32)>

Stores the image, resets pan, resets animation, updates the navigator, starts preloading. Returns Some((w,h)) — a return value, not a side-effect.

5.9 fn clear_image()

Resets current_image and current_path, then delegates to reset_pan() and reset_animation().

5.10 fn reset_animation() / fn tick_animation()

reset_animation() sets frame_index=0 and computes next_frame_at. Called from both apply_image() and clear_image().

tick_animation() checks the deadline, advances frame_index, schedules the next deadline, uses if-let — no unwrap().

5.11 fn reset_pan() / fn start_drag() / fn update_drag() / fn end_drag()

reset_pan() zeroes pan and clears drag_start. Called on every new image and on Backspace/Home/middle-click.

update_drag() computes new pan from cursor delta. Returns true if dragging so the caller can request a redraw without a separate is_some() check.

5.12 fn navigate(forward: bool)

Calls navigator.next() or prev() — both return None when fewer than 2 images exist, making navigation on a single-image directory a no-op.

5.13 fn request_delete() / fn on_deleted()

request_delete() sets deleting=true and spawns a thread for remove_file(). The UI never blocks on deletion I/O.

on_deleted() clears the flag, removes from navigator, clears the preload cache, navigates to the next sibling or clears the view.

5.14 fn drain_messages() → Option<(u32, u32)>

Non-blocking try_recv() loop. Stale Loaded messages are discarded. Returns resize dimensions as a return value.

5.15 Rendering functions

- **fit_scale()** — min(win_h/img_h, win_w/img_w). Always used since zoom is always 1.0.
- **nearest()** — point sample at integer-truncated source coordinate. Fast and correct at fit-to-window scale.
- **composite()** — alpha-blends a straight-RGBA pixel over BG. Inner blend() shares the formula across R, G, B.
- **render()** — takes Option<View>. Always paints unconditionally — no dirty flag — so off-screen regions paint correctly when the window is partially off-screen.

5.16 fn to_wide() / fn confirm_delete_dialog() / fn show_error_dialog()

to_wide() is a shared helper that encodes a Rust &str as a null-terminated UTF-16 Vec for Win32 API calls. Previously the encode_wide().chain(Some(0)).collect() pattern was duplicated four times across the two dialog functions. The helper eliminates that repetition.

Both dialogs call **MessageBoxW** — blocking, correct for modal confirmation. #[cfg(not(windows))] fallbacks for cross-platform compilation.

5.17 Window helpers

work_area() / **windows_work_area()** — usable desktop dimensions. SPI_GETWORKAREA on Windows; 90%×85% heuristic elsewhere.

clamp_to_work_area() — scales image dimensions to fit the usable area. **center_window()** — positions the window at the monitor centre with a minimum Y offset so the title bar is always reachable.

5.18 fn load_window_icon()

Loads the window title bar and taskbar icon. devoid.ico is embedded at compile time via include_bytes!("../devoid.ico") — no runtime file path needed. The image crate decodes the ICO (which contains 16×16, 32×32, 48×48, and 256×256 frames) and the best-fitting frame is passed to winit as RGBA bytes via winit::window::Icon::from_rgba(). The icon is set on the WindowBuilder via with_window_icon(), which sets both the title bar icon and the taskbar button icon.

> ■ *devoid.ico must contain multiple sizes. A single 256×256 image produces a blocky distorted result in the 16×16 title bar slot because Windows scales it with nearest-neighbour. The ico file included in the project was generated with Lanczos downscaling from the original 256×256 source.*

5.19 fn main() — startup sequence

- **image_size_hint()** reads only the file header for dimensions (no full decode). For SVG it parses the full document since there is no header-only path.
- **EventLoopBuilder type annotation** — let mut el_builder: EventLoopBuilder<WakeUp> instead of a turbofish. Same meaning, cleaner syntax.
- **WM_SYSCOMMAND/SC_CLOSE hook** — on Windows, the taskbar thumbnail close button sends WM_SYSCOMMAND/SC_CLOSE which winit 0.29 does not translate into CloseRequested. EventLoopBuilderExtWindows::with_msg_hook intercepts this message and re-posts it as WM_CLOSE, which winit does handle. Both close paths (real window X button and taskbar

thumbnail) now go through the same CloseRequested handler.

- **with_visible(!start_hidden)** — window is hidden when an image argument is provided. Revealed after resize and center are applied. Visible immediately when started without an argument.
- **pack_rgb(BG, BG, BG) in the pre-loop gray paint** — previously the gray value was computed inline as a manual bit shift. Now calls pack_rgb() consistently.
- **if start_hidden for clamp/center** — previously used if initial_path.is_some() which was redundant with start_hidden defined one line earlier.
- **RedrawRequested always paints** — no dirty flag. Prevents stale pixels in off-screen regions.
- **GIF control flow** — ControlFlow::WaitUntil when animating; ControlFlow::Wait otherwise. Zero CPU when idle.

6. Threading & Asynchronous Design

6.1 Why threads?

Image decoding is I/O-bound and CPU-bound. A large image can take hundreds of milliseconds. Doing this on the event loop thread freezes the window.

6.2 The channel pipeline

- One bounded crossbeam channel, capacity 32.
- msg_tx cloned for each background thread — cheap.
- msg_rx drained in drain_messages() on each WakeUp.
- Background threads post a Msg then call proxy.send_event(WakeUp).

■ *proxy.send_event() is thread-safe — EventLoopProxy is Arc-based.*

6.3 ControlFlow::Wait — zero idle CPU

The event loop sleeps until an event arrives. ControlFlow::WaitUntil wakes precisely at the next GIF frame deadline.

6.4 Header probe + async decode

image_size_hint() reads only the file header. The window appears immediately at the correct size with gray background. The full decode runs on a background thread. This keeps startup fast even on cold Windows process starts.

6.5 Window reveal sequence

When an image argument is provided: window is hidden → gray paint → decode thread runs → WakeUp arrives → resize issued → pending_center=true → next WakeUp: center_window() → set_visible(true) → RedrawRequested paints the image.

6.6 Taskbar thumbnail close button

The taskbar thumbnail close button sends WM_SYSCOMMAND/SC_CLOSE. Winit 0.29 handles WM_CLOSE but not WM_SYSCOMMAND/SC_CLOSE. The msg_hook intercepts SC_CLOSE and re-posts WM_CLOSE to the same window via PostMessageW. Both close paths then reach the same CloseRequested handler.

6.7 Stale result discarding

Loaded messages are checked against loading_path. Mismatches are discarded.

6.8 Async deletion

remove_file() runs on a background thread. The deleting flag prevents double-deletion.

6.9 Off-screen pixel correctness

RedrawRequested always paints — no dirty flag. When the OS exposes a region that was clipped because the window was partially off-screen, the next RedrawRequested writes fresh pixels for the entire window.

6.10 The two-event resize/centre sequence

window.request_inner_size() is asynchronous. Set pending_center=true on the WakeUp that issues the resize. On the next WakeUp the resize has landed and center_window() is accurate.

7. Key Design Decisions & Trade-offs

winit + softbuffer over egui/eframe

~50-70 crates and ~2-3 MB vs ~200+ crates and ~9 MB. Trade-off: no widgets, native OS dialogs required.

Frame and DecodedImage in loader.rs

Eliminates the RawImage→DecodedImage conversion that existed when types were split across files.

No zoom, nearest-neighbour only

Scroll-wheel zoom was removed at the user's request. Nearest-neighbour is correct and fast at fit-to-window scale.

Single-image navigation is a no-op

nav.rs next() and prev() return None when files.len() < 2.

No dirty flag — always paint on RedrawRequested

Prevents stale off-screen pixels when the window is partially off-screen. Safe with ControlFlow::Wait since RedrawRequested only fires when explicitly requested or when the OS needs a repaint.

to_wide() helper for Win32 UTF-16 encoding

The encode_wide().chain(Some(0)).collect() pattern was duplicated four times. One helper eliminates the repetition.

pack_rgb() used consistently

The pre-loop gray paint previously computed the BG colour as a manual bit shift. Now calls pack_rgb(BG, BG, BG) like everything else.

include_bytes! for the window icon

devoid.ico is embedded at compile time. No runtime file path, no missing-file failure in production. The image crate decodes it — already a dependency, no extra crate needed.

Multi-size ICO file

The original icon was a single 256×256 image. Windows scales it to 16×16 with nearest-neighbour for the title bar, producing a blocky result. The ICO now contains 16×16, 32×32, 48×48, and 256×256 frames generated with Lanczos downscaling. Windows picks the closest size.

WM_SYSCOMMAND/SC_CLOSE re-post

The simplest correct fix for the taskbar thumbnail close bug. No new Msg variants, no atomic flags, no shared state — just re-post WM_CLOSE in the message hook and let winit's existing WM_CLOSE handler do the work.

Return values over side effects

apply_image() returns Option<(u32,u32)>. The App struct has no window-management fields except pending_center.

Native OS dialogs

MessageBoxW is blocking — correct for modal confirmation.

8. Caveats & Known Limitations

Startup white flash (minimised, not eliminated)

DWM fills every new window with white before the application paints. Mitigated by with_visible(false) and synchronous gray paint. The DWM white is never visible in normal operation.

Cold-start latency

On Windows, if the executable hasn't been run for a long time, the OS may have evicted the binary from the page cache. Reload from disk, Authenticode verification, and CRT initialisation can take up to a minute on slow storage. This is a Windows OS behaviour, not a code issue. Subsequent runs are fast.

No status bar

Filename, index, and dimensions are not displayed. Requires a font rasteriser crate.

Animated GIF only

APNG and animated WebP/AVIF would need separate decoder paths.

Render loop is O(win_w × win_h)

Iterates every window pixel on every redraw. Could be parallelised with rayon.

No pan persistence across images

Pan resets to centre on every image navigation.

JXL HDR clamped, not tone-mapped

f32_to_u8 clamps values above 1.0.

SVG external resources not loaded

usvg with default options does not resolve external fonts or linked images.

monitor_usable heuristic on non-Windows

The SPI_GETWORKAREA path is Windows-only. Other platforms use 90%×85%.

9. Installer — main.wxs

Built with WiX 3 via cargo-wix. Run cargo wix from the project root to produce an MSI in target\wix\.

9.1 Product identity

The **UpgradeCode** GUID is fixed and must never change. **Product Id='*'** generates a new GUID on every build. **MajorUpgrade** removes the old version automatically.

9.2 Codepage constraints

WiX 3 does not support UTF-8 (65001) in Product/@Codepage or Package/@SummaryCodepage. Both are set to 1252 and the manufacturer name is ASCII-transliterated ('Hizlikan'). WiX 4 handles UTF-8 correctly but cargo-wix only supports WiX 3.

9.3 File associations — explicit RegistryValue elements

WiX 3's built-in ProgId/Extension elements produce duplicate primary keys (LGHT0130) when registering multiple extensions under one ProgId. This is a known WiX 3 bug. The workaround is to write all registry entries as explicit RegistryValue elements.

9.4 Three installer features

Feature	Default	What it does
Add to Open With	✓ On	Adds Devoid Viewer to the right-click Open With menu for all supported formats witho
Set as Default Image Viewer	✓ On	Makes Devoid Viewer the default opener for all supported formats.
Desktop Shortcut	✓ On	Places a shortcut on the desktop.

9.5 Registry structure

- **ProgIdRegistration** — HKCR\DevoidViewer.ImageFile (display name), shell\open\command (launch command), DefaultIcon. Always installed.
- **OpenWithAssociations** — HKCR\.ext\OpenWithProgIds\DevoidViewer.ImageFile for each of the 11 supported extensions.
- **DefaultAssociations** — HKCR\.ext = DevoidViewer.ImageFile for each extension, making Devoid Viewer the default opener.

9.6 Shell icon cache refresh

A CustomAction runs ie4uinit.exe -ClearIconCache after install and uninstall. Execute='immediate' — no rollback requirement.

9.7 Building the installer

- Install WiX Toolset 3.x and cargo-wix (cargo install cargo-wix).
- cargo wix — builds the release binary and MSI.

- cargo wix --nocapture — shows full output for debugging.

10. Glossary of Rust Concepts Used

Ownership
Every value has exactly one owner. When the owner goes out of scope the value is dropped. No garbage collector needed.

Borrowing
Lend a reference (&T;) without transferring ownership. References cannot outlive the value they point to.

&mut; self vs &self;
&mut; self = exclusive mutable access. &self; = shared read-only access. The compiler enforces that both cannot coexist.

Option
Either Some(value) or None. Safe replacement for null pointers.

Result
Either Ok(value) or Err(error). The ? operator propagates errors automatically.

Vec
A growable heap-allocated array.

HashMap
A hash table. O(1) average lookup, insert, and delete.

Arc
Atomically Reference Counted smart pointer. Shared ownership across threads.

Enum with data
Rust enums carry data per variant. Msg::Loaded carries path, image, bool. Unlike C enums which carry no data.

From / Into
Standard conversion traits. impl From for Y lets you write x.into().

then_some
(condition).then_some(value) returns Some(value) if true, None otherwise.

Lifetime
A compile-time label on a reference. View<'a> borrows from App — the compiler ensures View cannot outlive App.

Closure
An anonymous function. move |event, elwt| {...} moves ownership of captured variables — required for event_loop.run() and with_msg_hook().

match
Exhaustive pattern matching. Every variant must be handled.

#[derive(...)]
Auto-generates trait implementations. #[derive(Clone, Debug)] writes clone() and the debug formatter.

Send
Marker trait: safe to transfer to another thread. Derived automatically for types containing only Send fields.

ControlFlow
Winit enum. Wait sleeps until an event. WaitUntil sleeps until a deadline.

EventLoopProxy
Thread-safe handle to the event loop. proxy.send_event() injects a synthetic event from any thread.

NonZeroU32
A u32 guaranteed non-zero at compile time. softbuffer requires this for resize.

cfg(windows)
Conditional compilation. Code in #[cfg(windows)] blocks compiles on Windows only.

unsafe
Allows calling C API functions — MessageBoxW, SystemParametersInfoW, PostMessageW.

type alias
type Proxy = ...; creates a shorter name with no runtime cost.

Zero-size struct
struct WakeUp; has no fields and zero runtime size. Used as a type-level marker.

include_bytes!
Embeds a file's contents as a byte array at compile time. Used for devoid.ico so the icon is always available regardless of working directory.

msg_hook
winit 0.29 Windows platform extension. A callback receiving every Win32 message before winit processes it. Used to intercept WM_SYSCOMMAND/SC_CLOSE and re-post as WM_CLOSE.